Untitled

Samika Reddy

2026-04-19

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see http://rmarkdown.rstudio.com.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

```r
summary(cars)
```

```
##      speed           dist       
##  Min.   : 4.0   Min.   :  2.00  
##  1st Qu.:12.0   1st Qu.: 26.00  
##  Median :15.0   Median : 36.00  
##  Mean   :15.4   Mean   : 42.98  
##  3rd Qu.:19.0   3rd Qu.: 56.00  
##  Max.   :25.0   Max.   :120.00  
```

Including Plots

You can also embed plots, for example:

“‘{r}—

Baby Example: Predicting MPG with `mtcars`

To make BART more concrete, consider the `mtcars` dataset.
Suppose we want to predict a car’s fuel efficiency, measured by miles per gallon (`mpg`), using the other car characteristics such as horsepower, weight, number of cylinders, and transmission type.

This is a regression problem because the response variable, `mpg`, is continuous.

A simple baseline method for this problem is **linear regression**, which assumes the relationship between `mpg` and the predictors is approximately linear.
However, in practice, relationships between fuel efficiency and car features may be more complex and nonlinear.

This is where **Bayesian Additive Regression Trees (BART)** can help.
Instead of fitting one straight-line relationship, BART fits the response as a **sum of many small trees**, allowing it to capture nonlinearities and interactions automatically.

Step 1: Load the Data and Define the Problem

```r
# Load the BART package
library(BART)
```

```
## Loading required package: nlme

## Loading required package: survival
```

```r
# Load the built-in mtcars dataset
data(mtcars)

# Look at the first few rows
head(mtcars)
```

```
##                    mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
## Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
## Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
## Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
```

```r
# Our goal is to predict mpg
# So mpg will be the response variable y
y_reg <- mtcars$mpg

# Use all remaining variables as predictors
# BART wants predictors in matrix form
x_reg <- as.matrix(mtcars[, -1])

# Check dimensions
dim(x_reg)
```

```
## [1] 32 10
```

```r
length(y_reg)
```

```
## [1] 32
```

```r
# Set a seed so the results are reproducible
set.seed(123)

# Fit the BART regression model
# wbart() is used for continuous outcomes
bart_reg <- wbart(x.train = x_reg, y.train = y_reg)
```

```
## *****Into main of wbart
## *****Data:
## data:n,p,np: 32, 10, 0
## y1,yn: 0.909375, 1.309375
## x1,x[n*p]: 6.000000, 2.000000
## *****Number of Trees: 200
## *****Number of Cut Points: 2 ... 5
## *****burn and ndpost: 100, 1000
## *****Prior:beta,alpha,tau,nu,lambda: 2.000000,0.950000,0.415425,3.000000,1.368126
## *****sigma: 2.650197
## *****w (weights): 1.000000 ... 1.000000
## *****Dirichlet:sparse,theta,omega,a,b,rho,augment: 0,0,1,0.5,1,10,0
```

```
## *****nkeeptrain,nkeeptest,nkeeptestme,nkeeptreedraws: 1000,1000,1000,1000
## *****printevery: 100
## *****skiptr,skipte,skipteme,skiptreedraws: 1,1,1,1
##
## MCMC
## done 0 (out of 1100)
## done 100 (out of 1100)
## done 200 (out of 1100)
## done 300 (out of 1100)
## done 400 (out of 1100)
## done 500 (out of 1100)
## done 600 (out of 1100)
## done 700 (out of 1100)
## done 800 (out of 1100)
## done 900 (out of 1100)
## done 1000 (out of 1100)
## time: 1s
## check counts
## trcnt,tecnt,temecnt,treedrawscnt: 1000,0,0,1000
```

```r
# Print the model object

# Extract the average fitted values from BART
# These are the predicted mpg values
bart_pred_reg <- bart_reg$yhat.train.mean

# Compare actual mpg and predicted mpg
head(data.frame(
  Actual_MPG = y_reg,
  BART_Predicted_MPG = bart_pred_reg
))
```

```
##   Actual_MPG BART_Predicted_MPG
## 1       21.0           20.70781
## 2       21.0           20.64911
## 3       22.8           25.15813
## 4       21.4           20.02790
## 5       18.7           17.42987
## 6       18.1           19.40482
```

```r
# Extract the average fitted values from BART
# These are the predicted mpg values
bart_pred_reg <- bart_reg$yhat.train.mean

# Compare actual mpg and predicted mpg
head(data.frame(
  Actual_MPG = y_reg,
  BART_Predicted_MPG = bart_pred_reg
))
```

```
##   Actual_MPG BART_Predicted_MPG
## 1       21.0           20.70781
## 2       21.0           20.64911
## 3       22.8           25.15813
## 4       21.4           20.02790
## 5       18.7           17.42987
```

```
## 6       18.1           19.40482
```

```r
# Fit a standard linear regression model
lm_model <- lm(mpg ~ ., data = mtcars)

# Get predicted values from the linear model
lm_pred <- predict(lm_model)

# Show the first few predictions
head(data.frame(
  Actual_MPG = y_reg,
  LM_Predicted_MPG = lm_pred
))
```

```
##                   Actual_MPG LM_Predicted_MPG
## Mazda RX4               21.0         22.59951
## Mazda RX4 Wag           21.0         22.11189
## Datsun 710              22.8         26.25064
## Hornet 4 Drive          21.4         21.23740
## Hornet Sportabout       18.7         17.69343
## Valiant                 18.1         20.38304
```

```r
# Compute mean squared error (MSE) for BART
bart_mse <- mean((y_reg - bart_pred_reg)^2)

# Compute mean squared error (MSE) for linear regression
lm_mse <- mean((y_reg - lm_pred)^2)

# Print both MSE values
bart_mse
```

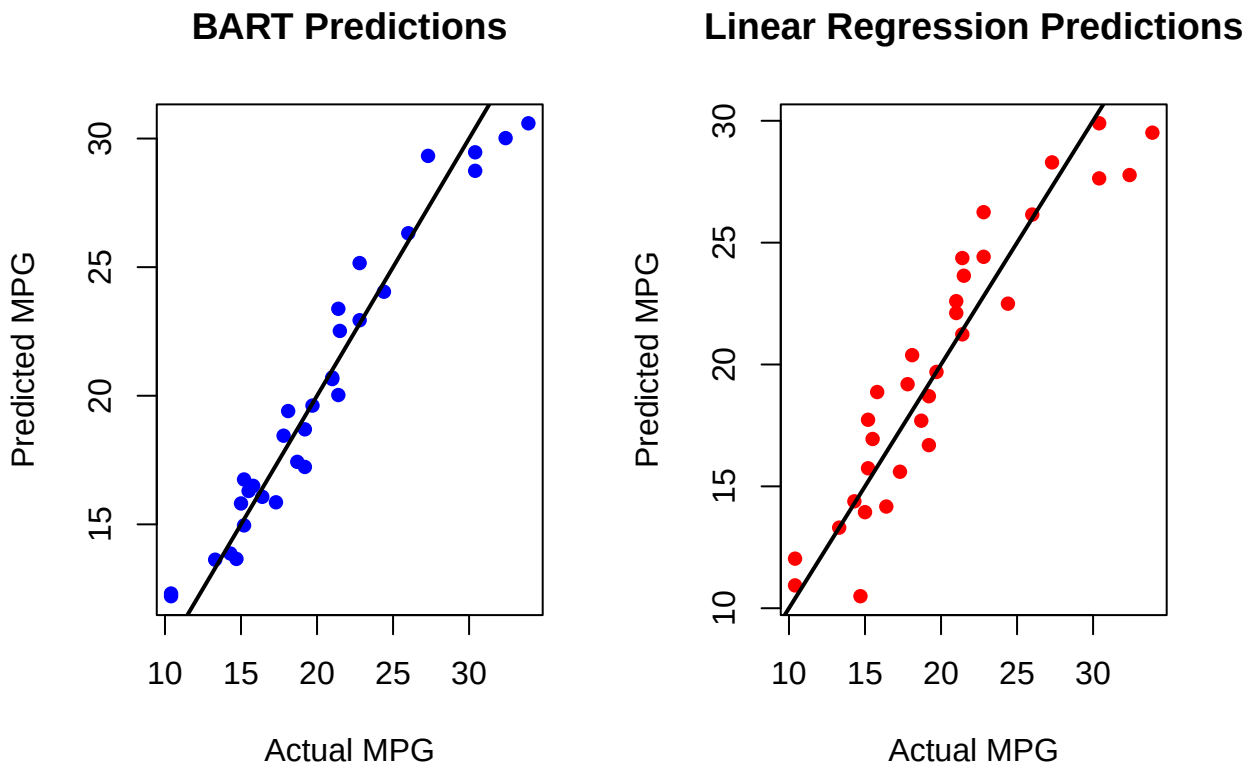
```
## [1] 1.868615
```

```r
lm_mse
```

```
## [1] 4.609201
```

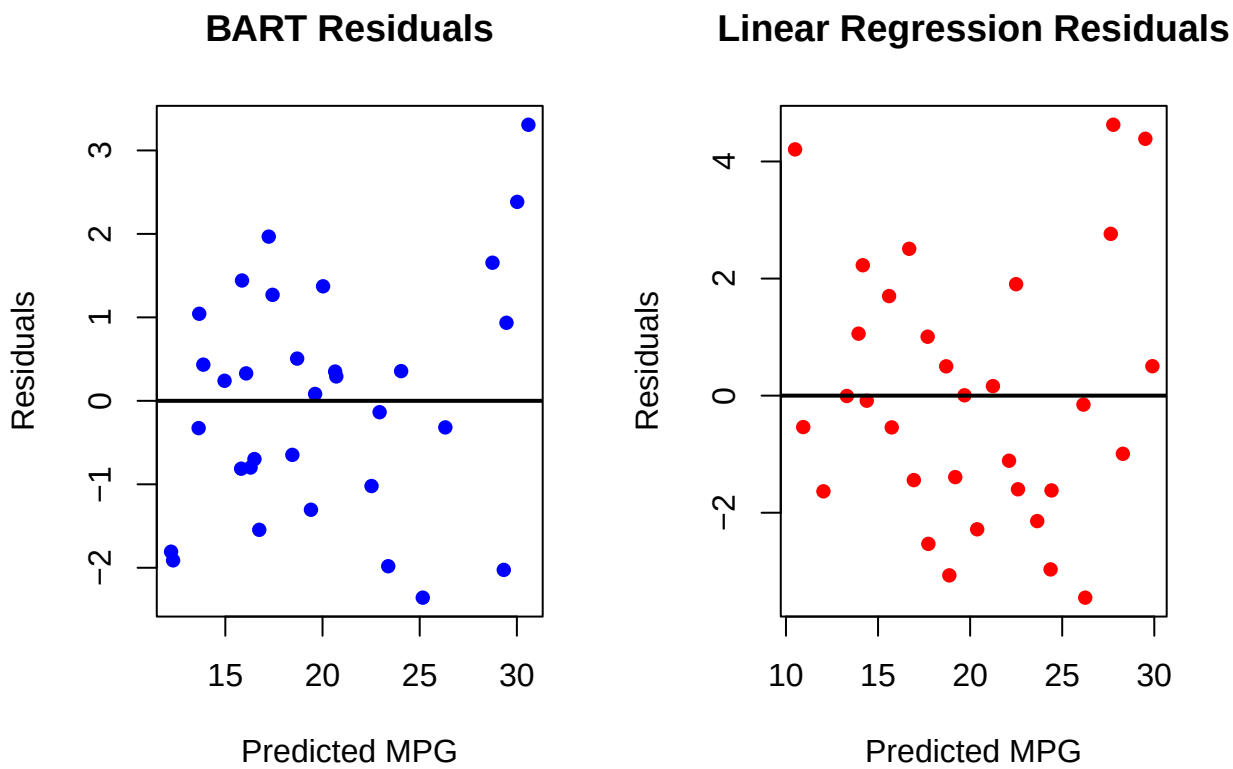
```r
# Show the two models side-by-side
par(mfrow = c(1, 2))

# Plot actual vs predicted for BART
plot(y_reg, bart_pred_reg,
     col = "blue",
     pch = 16,
     xlab = "Actual MPG",
     ylab = "Predicted MPG",
     main = "BART Predictions")

# Add the 45-degree reference line
abline(0, 1, lwd = 2)

# Plot actual vs predicted for linear regression
plot(y_reg, lm_pred,
     col = "red",
     pch = 16,
     xlab = "Actual MPG",
     ylab = "Predicted MPG",
```

```r
     main = "Linear Regression Predictions")

# Add the same 45-degree line
abline(0, 1, lwd = 2)
```




```r
# Reset plotting layout
par(mfrow = c(1, 1))



# Residuals for BART
bart_resid <- y_reg - bart_pred_reg

# Residuals for linear regression
lm_resid <- y_reg - lm_pred

# Plot residuals side-by-side
par(mfrow = c(1, 2))

plot(bart_pred_reg, bart_resid,
     col = "blue",
     pch = 16,
     xlab = "Predicted MPG",
     ylab = "Residuals",
     main = "BART Residuals")
abline(h = 0, lwd = 2)

plot(lm_pred, lm_resid,
     col = "red",
     pch = 16,
```

```r
     xlab = "Predicted MPG",
     ylab = "Residuals",
     main = "Linear Regression Residuals")
abline(h = 0, lwd = 2)
```




```r
# Reset plotting layout
par(mfrow = c(1, 1))
```

Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

Baby Example: A Nonlinear Regression Example

To see a more dramatic comparison, consider a simulated dataset where the true relationship is clearly nonlinear.

Suppose the response is generated by:

$$y = \sin(x) + \epsilon$$

where ϵ is random noise.

In this setting, **linear regression** will struggle because it tries to fit a straight line, while **BART** can adapt to the curved relationship much more effectively.

Step 1: Simulate the Data

```r
# Load BART
library(BART)

# Set seed for reproducibility
set.seed(123)

# Simulate predictor values
x <- seq(0, 10, length.out = 200)

# Simulate nonlinear response
y <- sin(x) + rnorm(200, sd = 0.2)

# Put into a data frame for convenience
sim_data <- data.frame(x = x, y = y)

# Look at first few rows
head(sim_data)
```

```
##            x            y
## 1 0.00000000 -0.112095129
## 2 0.05025126  0.004194612
## 3 0.10050251  0.412075069
## 4 0.15075377  0.164285073
## 5 0.20100503  0.225511769
## 6 0.25125628  0.591633988
```

```r
# BART needs predictor matrix form
x_mat <- as.matrix(sim_data$x)

# Fit BART regression model
bart_model <- wbart(x.train = x_mat, y.train = sim_data$y)
```

```
## *****Into main of wbart
## *****Data:
## data:n,p,np: 200, 1, 0
## y1,yn: -0.291970, -0.960992
## x1,x[n*p]: 0.000000, 10.000000
## *****Number of Trees: 200
## *****Number of Cut Points: 100 ... 100
## *****burn and ndpost: 100, 1000
## *****Prior:beta,alpha,tau,nu,lambda: 2.000000,0.950000,0.048843,3.000000,0.093819
## *****sigma: 0.694002
## *****w (weights): 1.000000 ... 1.000000
## *****Dirichlet:sparse,theta,omega,a,b,rho,augment: 0,0,1,0.5,1,1,0
## *****nkeeptrain,nkeeptest,nkeeptestme,nkeeptreedraws: 1000,1000,1000,1000
## *****printevery: 100
## *****skiptr,skipte,skipteme,skiptreedraws: 1,1,1,1
##
## MCMC
## done 0 (out of 1100)
## done 100 (out of 1100)
## done 200 (out of 1100)
## done 300 (out of 1100)
```

```
## done 400 (out of 1100)
## done 500 (out of 1100)
## done 600 (out of 1100)
## done 700 (out of 1100)
## done 800 (out of 1100)
## done 900 (out of 1100)
## done 1000 (out of 1100)
## time: 1s
## check counts
## trcnt,tecnt,temecnt,treedrawscnt: 1000,0,0,1000
```

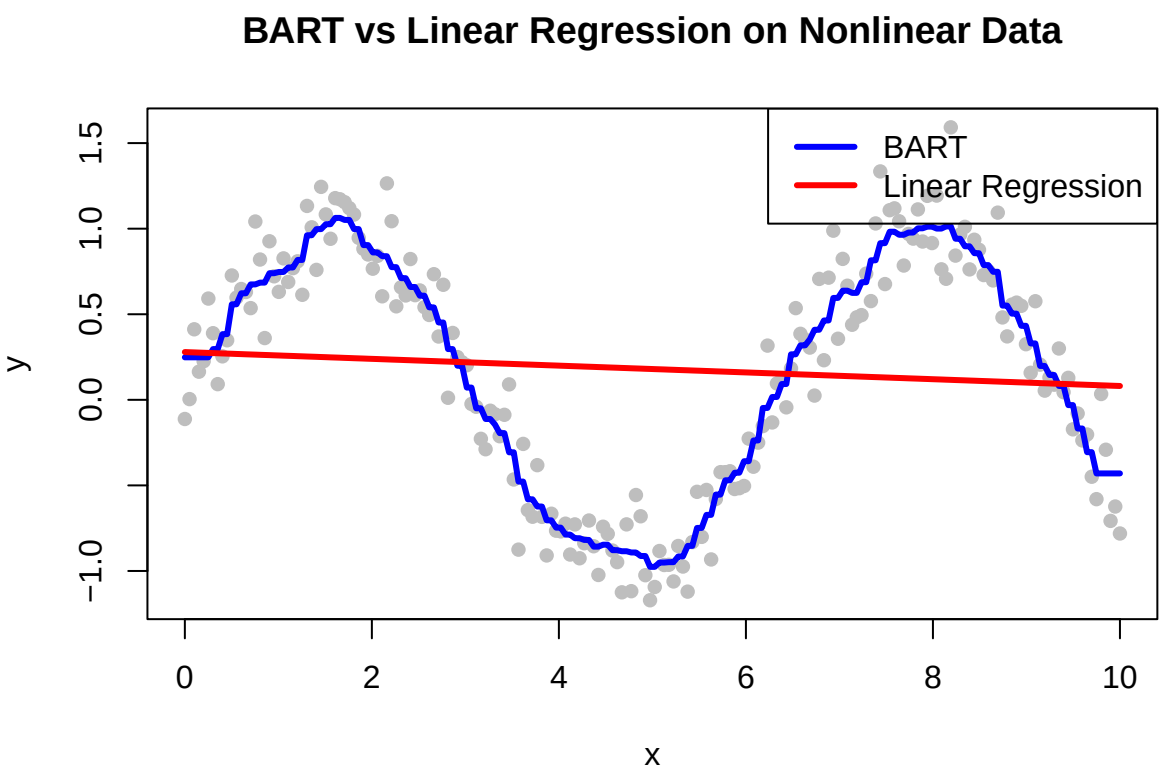
```r
# Get BART fitted values
bart_pred <- bart_model$yhat.train.mean

# Fit linear regression
lm_model <- lm(y ~ x, data = sim_data)

# Get linear regression predictions
lm_pred <- predict(lm_model)

# Compute mean squared errors
bart_mse <- mean((sim_data$y - bart_pred)^2)
lm_mse <- mean((sim_data$y - lm_pred)^2)

bart_mse
```

```
## [1] 0.03006255
```

```r
lm_mse
```

```
## [1] 0.4768228
```

```r
# Plot the observed data
plot(sim_data$x, sim_data$y,
     pch = 16,
     col = "gray",
     xlab = "x",
     ylab = "y",
     main = "BART vs Linear Regression on Nonlinear Data")

# Add BART fitted curve
lines(sim_data$x, bart_pred,
      col = "blue",
      lwd = 3)

# Add linear regression fitted line
lines(sim_data$x, lm_pred,
      col = "red",
      lwd = 3)

# Add legend
legend("topright",
       legend = c("BART", "Linear Regression"),
       col = c("blue", "red"),
       lwd = 3)
```




```
### Why this one is better
#- the visual difference is much stronger
#- BART's curve will usually look clearly better
#- the MSE gap is often larger than in `mtcars`

### What to say
#> In this example, the true relationship is nonlinear, so #linear regression is misspecified. BART per

#If you want, I can also give you a **classification example #with more dramatic results** too.
```

Baby Example: Nonlinear Classification (BART vs GLM)

#To better illustrate the advantage of BART, consider a #simulated dataset with a **nonlinear decision boundary**.

#In this setting: #- GLM (logistic regression) assumes a linear boundary #- BART can model complex nonlinear patterns

---

```
library(BART)

set.seed(123)

# Generate data
n <- 200
x1 <- runif(n, -3, 3)
x2 <- runif(n, -3, 3)

# Nonlinear boundary (circle-like pattern)
```

```r
y <- ifelse(x1^2 + x2^2 < 4, 1, 0)

sim_data <- data.frame(x1 = x1, x2 = x2, y = y)

head(sim_data)
```

```
##           x1          x2 y
## 1 -1.2745349 -1.56764384 0
## 2  1.7298308  2.77415362 0
## 3 -0.5461385  0.60819436 1
## 4  2.2981044  0.09017836 0
## 5  2.6428037 -0.58455995 0
## 6 -2.7266610  2.28147925 0
```

```r
# BART model
x_mat <- as.matrix(sim_data[, c("x1", "x2")])
y_vec <- sim_data$y

set.seed(123)
bart_model <- pbart(x.train = x_mat, y.train = y_vec)
```

```
## *****Into main of pbart
## *****Data:
## data:n,p,np: 200, 2, 0
## y1,yn: 0, 0
## x1,x[n*p]: -1.274535, 2.947419
## *****Number of Trees: 50
## *****Number of Cut Points: 100 ... 100
## *****burn and ndpost: 100, 1000
## *****Prior:mybeta,alpha,tau: 2.000000,0.950000,0.212132
## *****binaryOffset: -0.412463
## *****Dirichlet:sparse,theta,omega,a,b,rho,augment: 0,0,1,0.5,1,2,0
## *****nkeeptrain,nkeeptest,nkeeptreedraws: 1000,1000,1000
## *****printevery: 100
## *****skiptr,skipte,skiptreedraws: 1,1,1
##
## MCMC
## done 0 (out of 1100)
## done 100 (out of 1100)
## done 200 (out of 1100)
## done 300 (out of 1100)
## done 400 (out of 1100)
## done 500 (out of 1100)
## done 600 (out of 1100)
## done 700 (out of 1100)
## done 800 (out of 1100)
## done 900 (out of 1100)
## done 1000 (out of 1100)
## time: 0s
## check counts
## trcnt,tecnt: 1000,0
```

```r
bart_prob <- bart_model$prob.train.mean
bart_pred <- ifelse(bart_prob > 0.5, 1, 0)
```

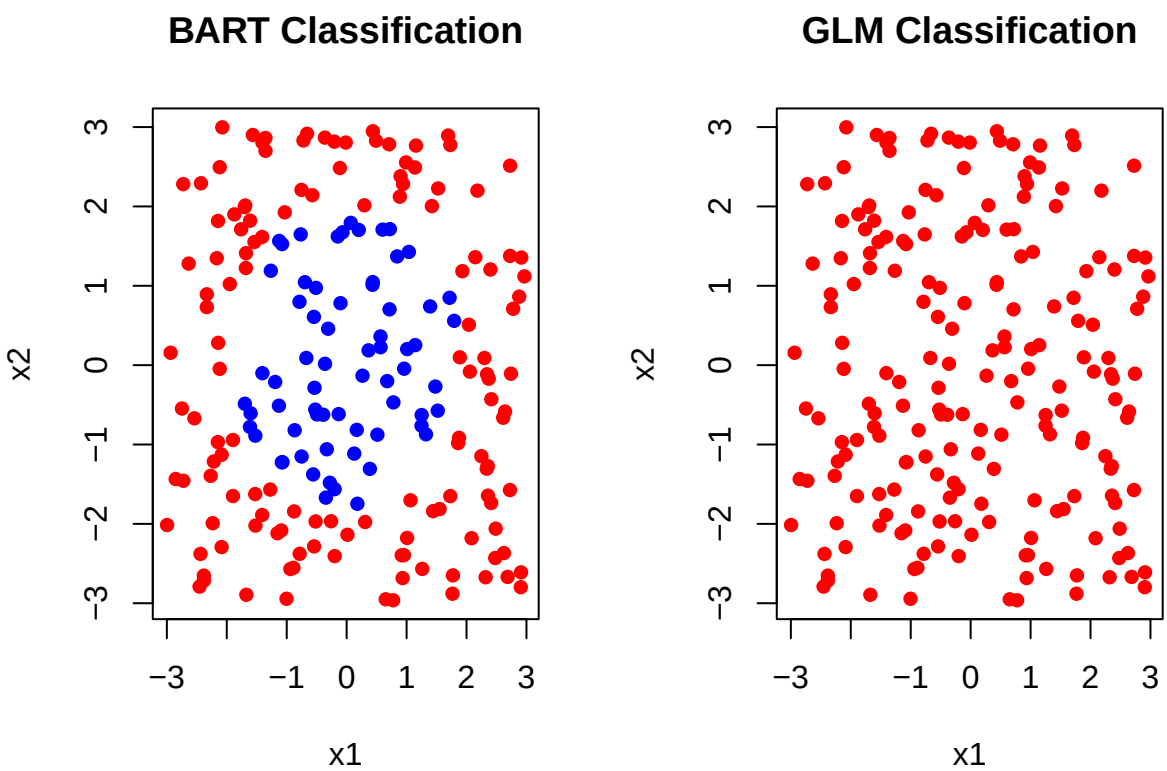
```r
# GLM (logistic regression)
glm_model <- glm(y ~ x1 + x2, data = sim_data, family = "binomial")
glm_prob <- predict(glm_model, type = "response")
glm_pred <- ifelse(glm_prob > 0.5, 1, 0)

# Accuracy comparison
bart_acc <- mean(bart_pred == y_vec)
glm_acc <- mean(glm_pred == y_vec)

bart_acc
```

```
## [1] 0.99
```

```r
glm_acc
```

```
## [1] 0.66
```

```r
par(mfrow = c(1,2))

# Plot BART predictions
plot(sim_data$x1, sim_data$x2,
     col = ifelse(bart_pred == 1, "blue", "red"),
     pch = 16,
     main = "BART Classification",
     xlab = "x1", ylab = "x2")

# Plot GLM predictions
plot(sim_data$x1, sim_data$x2,
     col = ifelse(glm_pred == 1, "blue", "red"),
     pch = 16,
     main = "GLM Classification",
     xlab = "x1", ylab = "x2")
```




```
par(mfrow = c(1,1))
```

Baby Example: BART vs Boosting (mtcars)

We use the `mtcars` dataset to compare: - **BART** - **Boosting (GBM)**

We evaluate both models using: - **Training error** - **Testing error**

This helps us understand how well each model generalizes.

Step 1: Train-Test Split

```
library(BART)
library(gbm)
```

```
## Loaded gbm 2.2.3
```

```
## This version of gbm is no longer under development. Consider transitioning to gbm3, https://github.c
```

```
set.seed(123)

data(mtcars)

# Create train/test split (70/30)
n <- nrow(mtcars)
train_idx <- sample(1:n, size = floor(0.7 * n))

train <- mtcars[train_idx, ]
```

```r
test  <- mtcars[-train_idx, ]

# Define predictors and response
x_train <- as.matrix(train[, -1])
y_train <- train$mpg

x_test <- as.matrix(test[, -1])
y_test <- test$mpg

# -----------------------------
# BART MODEL
# -----------------------------
bart_model <- wbart(x.train = x_train, y.train = y_train, x.test = x_test)
```

```
## *****Into main of wbart
## *****Data:
## data:n,p,np: 22, 10, 10
## y1,yn: -6.059091, -6.359091
## x1,x[n*p]: 8.000000, 4.000000
## xp1,xp[np*p]: 6.000000, 2.000000
## *****Number of Trees: 200
## *****Number of Cut Points: 2 ... 5
## *****burn and ndpost: 100, 1000
## *****Prior:beta,alpha,tau,nu,lambda: 2.000000,0.950000,0.415425,3.000000,1.947922
## *****sigma: 3.162284
## *****w (weights): 1.000000 ... 1.000000
## *****Dirichlet:sparse,theta,omega,a,b,rho,augment: 0,0,1,0.5,1,10,0
## *****nkeeptrain,nkeeptest,nkeeptestme,nkeeptreedraws: 1000,1000,1000,1000
## *****printevery: 100
## *****skiptr,skipte,skipteme,skiptreedraws: 1,1,1,1
##
## MCMC
## done 0 (out of 1100)
## done 100 (out of 1100)
## done 200 (out of 1100)
## done 300 (out of 1100)
## done 400 (out of 1100)
## done 500 (out of 1100)
## done 600 (out of 1100)
## done 700 (out of 1100)
## done 800 (out of 1100)
## done 900 (out of 1100)
## done 1000 (out of 1100)
## time: 1s
## check counts
## trcnt,tecnt,temecnt,treedrawscnt: 1000,1000,1000,1000
```

```r
bart_train_pred <- bart_model$yhat.train.mean
bart_test_pred  <- bart_model$yhat.test.mean

# -----------------------------
# BOOSTING MODEL
# -----------------------------
boost_model <- gbm(
```

```r
  mpg ~ .,
  data = train,
  distribution = "gaussian",
  n.trees = 200,
  interaction.depth = 2,
  shrinkage = 0.05,
  bag.fraction = 1.0,      # use all training rows each iteration
  n.minobsinnode = 2,      # smaller minimum node size
  verbose = FALSE
)

boost_train_pred <- predict(boost_model, train, n.trees = 200)
boost_test_pred  <- predict(boost_model, test, n.trees = 200)

# ----------------------------
# COMPARE TRAINING / TEST ERROR
# ----------------------------
bart_train_mse  <- mean((y_train - bart_train_pred)^2)
bart_test_mse   <- mean((y_test - bart_test_pred)^2)

boost_train_mse <- mean((y_train - boost_train_pred)^2)
boost_test_mse  <- mean((y_test - boost_test_pred)^2)

bart_train_mse
```

```
## [1] 3.049989
```

```r
bart_test_mse
```

```
## [1] 5.174602
```

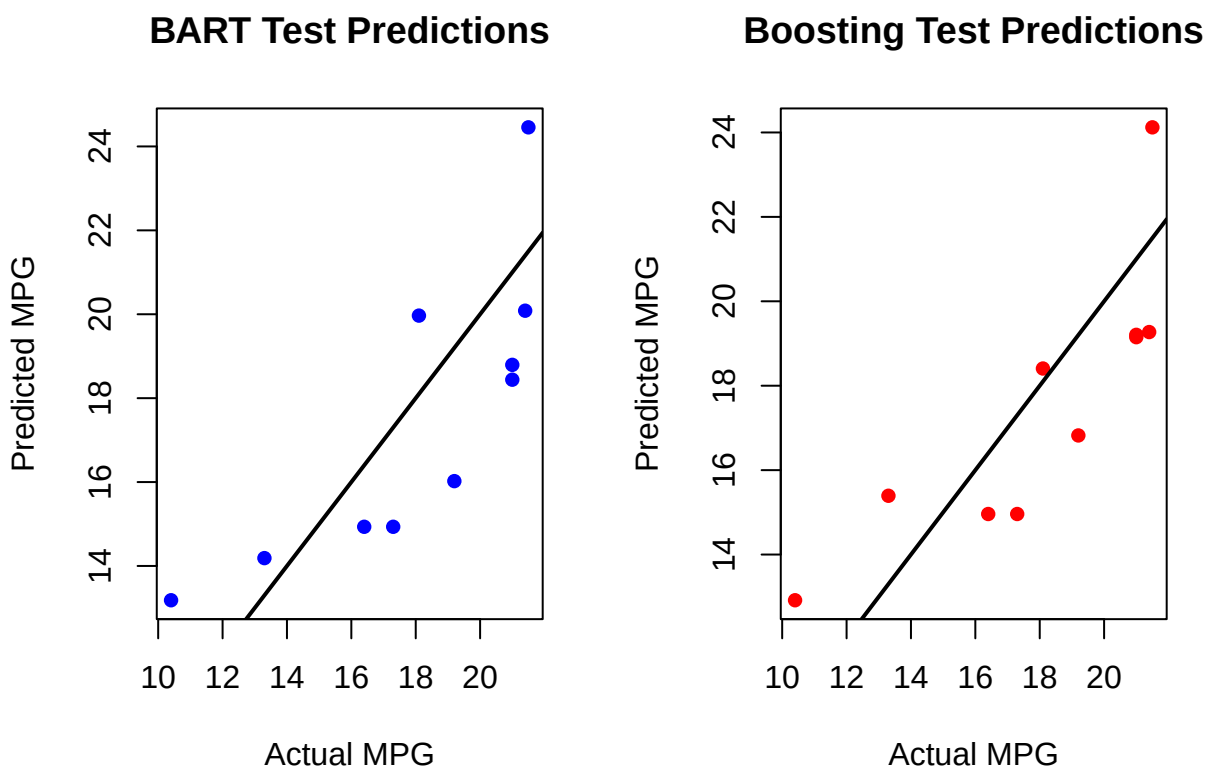
```r
boost_train_mse
```

```
## [1] 0.1846259
```

```r
boost_test_mse
```

```
## [1] 4.205866
```

```r
# ----------------------------
# TEST SET PLOTS
# ----------------------------
par(mfrow = c(1, 2))

plot(y_test, bart_test_pred,
     col = "blue", pch = 16,
     main = "BART Test Predictions",
     xlab = "Actual MPG",
     ylab = "Predicted MPG")
abline(0, 1, lwd = 2)

plot(y_test, boost_test_pred,
     col = "red", pch = 16,
     main = "Boosting Test Predictions",
     xlab = "Actual MPG",
     ylab = "Predicted MPG")
abline(0, 1, lwd = 2)
```




```r
par(mfrow = c(1, 1))
```

```r
# Create dataframe for plotting
error_df <- data.frame(
  Model = rep(c("BART", "Boosting"), each = 2),
  Type = rep(c("Train Error", "Test Error"), times = 2),
  MSE = c(bart_train_mse, bart_test_mse,
          boost_train_mse, boost_test_mse)
)

# Plot using ggplot
library(ggplot2)

ggplot(error_df, aes(x = Model, y = MSE, fill = Type)) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(title = "Training vs Testing Error Comparison",
       y = "Mean Squared Error") +
  theme_minimal()
```

```
library(BART)
library(gbm)

set.seed(123)

data(mtcars)

# Train/test split
n <- nrow(mtcars)
train_idx <- sample(1:n, size = floor(0.7 * n))

train <- mtcars[train_idx, ]
test  <- mtcars[-train_idx, ]

x_train <- as.matrix(train[, -1])
y_train <- train$mpg

x_test <- as.matrix(test[, -1])
y_test <- test$mpg

# ----------------------------
# BOOSTING: error over trees
# ----------------------------
n_trees <- 200

boost_model <- gbm(
  mpg ~ .,
```

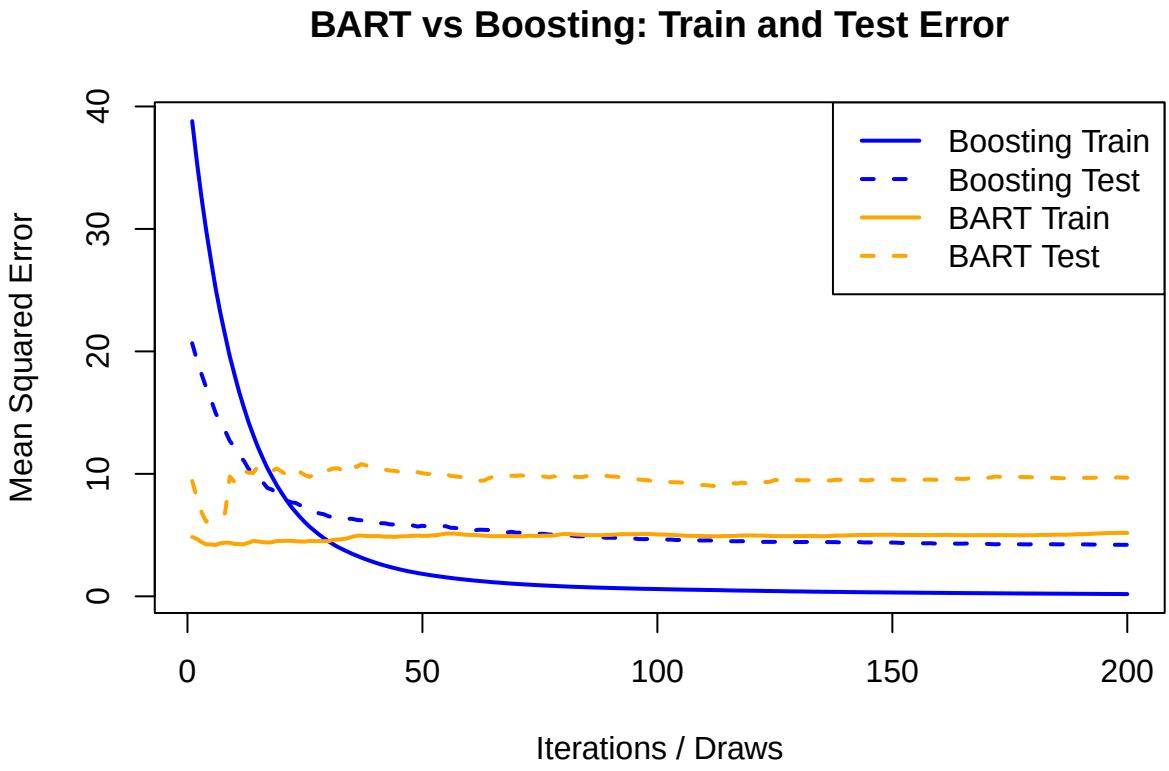
```r
  data = train,
  distribution = "gaussian",
  n.trees = n_trees,
  interaction.depth = 2,
  shrinkage = 0.05,
  bag.fraction = 1.0,
  n.minobsinnode = 2,
  verbose = FALSE
)

boost_train_err <- numeric(n_trees)
boost_test_err  <- numeric(n_trees)

for (i in 1:n_trees) {
  pred_train <- predict(boost_model, train, n.trees = i)
  pred_test  <- predict(boost_model, test,  n.trees = i)

  boost_train_err[i] <- mean((y_train - pred_train)^2)
  boost_test_err[i]  <- mean((y_test - pred_test)^2)
}

# ----------------------------
# BART: error over posterior draws
# ----------------------------
bart_model <- wbart(
  x.train = x_train,
  y.train = y_train,
  x.test  = x_test,
  ndpost  = 200
)
```

```
## *****Into main of wbart
## *****Data:
## data:n,p,np: 22, 10, 10
## y1,yn: -6.059091, -6.359091
## x1,x[n*p]: 8.000000, 4.000000
## xp1,xp[np*p]: 6.000000, 2.000000
## *****Number of Trees: 200
## *****Number of Cut Points: 2 ... 5
## *****burn and ndpost: 100, 200
## *****Prior:beta,alpha,tau,nu,lambda: 2.000000,0.950000,0.415425,3.000000,1.947922
## *****sigma: 3.162284
## *****w (weights): 1.000000 ... 1.000000
## *****Dirichlet:sparse,theta,omega,a,b,rho,augment: 0,0,1,0.5,1,10,0
## *****nkeeptrain,nkeeptest,nkeeptestme,nkeeptreedraws: 200,200,200,200
## *****printevery: 100
## *****skiptr,skipte,skipteme,skiptreedraws: 1,1,1,1
##
## MCMC
## done 0 (out of 300)
## done 100 (out of 300)
## done 200 (out of 300)
## time: 0s
## check counts
```

```
## trcnt,tecnt,temecnt,treedrawscnt: 200,200,200,200
```

```r
bart_train_mat <- bart_model$yhat.train
bart_test_mat  <- bart_model$yhat.test

# Make sure rows = posterior draws, columns = observations
if (ncol(bart_train_mat) == length(y_train)) {
  bart_train_draws <- bart_train_mat
} else {
  bart_train_draws <- t(bart_train_mat)
}

if (ncol(bart_test_mat) == length(y_test)) {
  bart_test_draws <- bart_test_mat
} else {
  bart_test_draws <- t(bart_test_mat)
}

n_draws <- nrow(bart_train_draws)

bart_train_err <- numeric(n_draws)
bart_test_err  <- numeric(n_draws)

for (i in 1:n_draws) {
  bart_train_err[i] <- mean((y_train - bart_train_draws[i, ])^2)
  bart_test_err[i]  <- mean((y_test - bart_test_draws[i, ])^2)
}

# -----------------------------
# Smooth BART errors
# -----------------------------
bart_train_err_smooth <- cumsum(bart_train_err) / seq_along(bart_train_err)
bart_test_err_smooth  <- cumsum(bart_test_err) / seq_along(bart_test_err)

# -----------------------------
# PLOT
# -----------------------------
plot(boost_train_err, type = "l",
     col = "blue", lwd = 2,
     ylim = range(c(boost_train_err, boost_test_err,
                    bart_train_err_smooth, bart_test_err_smooth)),
     xlab = "Iterations / Draws",
     ylab = "Mean Squared Error",
     main = "BART vs Boosting: Train and Test Error")

lines(boost_test_err, col = "blue", lty = 2, lwd = 2)
lines(bart_train_err_smooth, col = "orange", lwd = 2)
lines(bart_test_err_smooth, col = "orange", lty = 2, lwd = 2)

legend("topright",
       legend = c("Boosting Train", "Boosting Test",
                  "BART Train", "BART Test"),
       col = c("blue", "blue", "orange", "orange"),
       lty = c(1, 2, 1, 2),
```

```
        lwd = 2)
```


BART vs Boosting: Train and Test Error
Mean Squared Error
0
10
20
30
40
0
50
100
150
200
Iterations / Draws
Boosting Train
Boosting Test
BART Train
BART Test